



Deep Learning-based Sentiment Classification for Amazon Products Reviews

Data Driven Approach to Understand Sentiments of Product Reviews

IE7500

By:

Allison Selwah

Bowen Yang

Subhasish Bhaumik

Table of Contents

DEEP LEARNING-BASED SENTIMENT CLASSIFICATION FOR AMAZON PRODUCTS REVIEWS	1
PROJECT TITLE & DESCRIPTION	3
PROBLEM STATEMENT	3
DATA SOURCES	4
EXPECTED OUTCOMES.....	5
CODE REPO.....	5
MODEL BUILDING	5
<i>Data Pre-Processing</i>	5
<i>Model Training</i>	7
<i>Saving the Model</i>	8
MODEL EVALUATION.....	8
<i>Model Tracking</i>	8
<i>Evaluation Explanation</i>	9
<i>Additional Observations</i>	11
<i>Confusion Matrix</i>	11
<i>Error Analysis</i>	12
MODEL INFERENCE	13

Project Title & Description

Deep Learning-based Sentiment Classification for Amazon Product

This project aims to develop an advanced sentiment analysis model using Recurrent Neural Networks (RNN) and Long Short-Term Memory (LSTM) networks to classify product reviews from Amazon into positive and negative sentiment categories. By leveraging deep learning techniques, we will try to create a robust model capable of understanding the contextual details in product reviews, providing valuable insights into customer sentiment.

Problem Statement

In the digital age of e-commerce, product reviews have become a critical source of information for both consumers and businesses. The exponential growth of online shopping platforms has led to an unprecedented volume of user-generated content that holds immense strategic value. Consider these compelling insights:

1. Consumer Decision-Making:

According to [CapitalOne Shopping](#)

- 99% of consumers read online reviews before making a purchase and reviews influence 93% of consumers' purchasing decisions.
- Reviews influence purchasing decisions across all product categories
- Potential buyers rely heavily on sentiment expressed in previous customers' experiences

2. Business Intelligence:

- Companies lose approximately \$62 billion annually due to poor customer service(<https://www.forbes.com/sites/shephyken/2017/04/01/are-you-part-of-the-62-billion-loss-due-to-poor-customer-service/>)
- Sentiment analysis provides real-time insights into product performance and customer satisfaction
- Helps identify product strengths, weaknesses, and areas for improvement

3. Economic Impact:

- A single negative review can drive away potential customers
- Positive reviews can increase conversion rates by up to 3 times
- Reviews act as a crucial trust-building mechanism in online marketplaces

Challenges in Manual Review Analysis:

- Massive volume of reviews (millions generated daily)
- Time-consuming manual sentiment assessment
- Subjective human interpretation
- Inability to process reviews at scale

Relevance to NLP:

Recurrent Neural Networks (RNNs) are highly relevant for sentiment analysis in Natural Language Processing (NLP) due to their ability to effectively process sequential data, like text, where the meaning of a word can be influenced by the words that precede or follow it. Sentiment analysis involves determining the emotional tone behind a piece of text, such as whether a sentence expresses positive, negative, or neutral sentiment. RNNs are capable of capturing the temporal dependencies in text, allowing them to consider the context provided by earlier words when predicting sentiment.

- Modeling language's intrinsic sequentially
- Capturing complex contextual nuances
- Providing adaptive, learned representations
- Handling linguistic complexity dynamically

The approach transforms sentiment analysis from a static classification task to a sophisticated, context-aware computational linguistic challenge.

Data Sources

Dataset: Amazon Reviews Dataset from Kaggle

- Source: Kaggle (<https://www.kaggle.com/datasets/bittlingmayer/amazonreviews>)
- Structure:
 - Binary classification:
 - __label__1: 1-2 star reviews (negative sentiment)
 - __label__2: 4-5 star reviews (positive sentiment)
 - Excludes neutral 3-star reviews
 - Includes review titles and text
 - Primarily English language reviews, with some multilingual content

Volume:

- Training: 468 MB (3.6 million)
- Test: 52 MB (400K)

Data Structure:

Column	Description	Datatype
Label	Sentiment	Text
Reviews	Actual Product Reviews	Text

Expected Outcomes

Anticipated outcomes/learning

- Comparative performance analysis of RNN and LSTM models
- Understanding deep learning model's sentiment classification capabilities

Evaluation Process

We are planning to use Accuracy as an evaluation Metrics. Percentage of correctly classified sentiment labels (positive, negative, neutral) compared to the total number of samples.

But also, if we find the data to be imbalanced, we may also use Precision, Recall and F1-score. We also will produce a Confusion Matrix.

Code Repo

https://github.com/alliselwah/IE7500_GroupF.git

Model Building

Data Pre-Processing

Why Py-Spark?

PySpark was used for preprocessing this data because of the dataset's large size and the need for efficient, scalable processing. The training dataset contains 3.6 million reviews and the test set contains 400,000 reviews, totaling 4 million records. Processing such a volume of text data, involving steps like cleaning, tokenization, and applying UDFs, can be computationally expensive and memory-intensive for standard single-machine libraries like Pandas. PySpark, built on Apache Spark, is specifically designed for distributed data processing. It allows the workload to be parallelized across multiple CPU cores (as configured with local[*], executor instances, and cores) or even across a cluster of machines. This distributed approach significantly speeds up the preprocessing tasks and makes it feasible to handle datasets that might exceed the memory capacity of a single node, making PySpark a necessary tool for efficiently preparing this large-scale dataset for model training.

Processing Steps

1. Environment Setup:

- The process begins by setting up the Python environment for PySpark, ensuring the correct Python executable paths are specified using environment variables (PYSPARK_PYTHON, PYSPARK_DRIVER_PYTHON). Necessary libraries like findspark, pyspark.sql, nltk, and re are imported. Warnings are suppressed for cleaner output.

2. Spark Session Initialization:

- A SparkSession is created with the application name "Amazon Reviews Analysis".
- It's configured to run locally using all available cores (local [*]) and specific settings for executor instances, cores, memory, Python environment, and driver result size are applied to manage resources effectively for potentially large datasets. The default log level is set to WARN.

3. Data Loading:

- The raw training and testing data are read from text files (amazon_train_dataset/train.ft.txt and amazon_train_dataset/test.ft.txt) into Spark DataFrames (df_train, df_test). Each line in these files is initially read as a single string column named "value".

4. Data Shaping and Feature Extraction:

- A function shaping_datasets is defined and applied to both train and test DataFrames.
- This function parses each row in the "value" column, splitting it based on the label prefix (like "__label__1 "). It extracts the label and the review text into separate columns.
- The extracted labels ("__label__1", "__label__2") are mapped to numerical sentiment values (0 for negative, 1 for positive) in a new "sentiment" column.
- The function returns new DataFrames (df_train_p, df_test_p) containing only the "sentiment" and "review_text" columns.
- The structure and counts of these reshaped DataFrames are checked.

5. Natural Language Processing (NLP) Setup:

- Necessary NLTK resources (punkt for tokenization, stop-words corpus, wordnet for lemmatization, averaged_perceptron_tagger) are downloaded.
- A WordNetLemmatizer instance and a set of English stop-words are initialized for later use.

6. Text Cleaning and Preprocessing Functions:

- Two Python functions are defined for text processing:
 - clean_text: Converts input text to lowercase, uses regular expressions to remove special characters and numbers, and strips extra whitespace.
 - tokenize_and_preprocess: Takes cleaned text, tokenizes it using nltk.word_tokenize
- These Python functions are registered as PySpark User-Defined Functions (UDFs) to enable their application across the distributed DataFrame rows (clean_text_udf, tokenize_and_preprocess_udf).

7. Preprocessing Pipeline Execution:

- A function process_reviews_with_nltk orchestrates the application of the UDFs.
- It takes a DataFrame as input, applies clean_text_udf to the "review_text" column to create "cleaned_text".
- Then, it applies tokenize_and_preprocess_udf to "cleaned_text" to generate a list of "processed_tokens".
- Finally, it joins the tokens in "processed_tokens" back into a space-separated string, creating the "processed_text" column.
- This pipeline function is executed on both the shaped train and test DataFrames (df_train_p, df_test_p), resulting in train_processed_reviews_df and test_processed_reviews_df.

8. Saving Processed Data:

- The fully processed datasets (containing "cleaned_text" and "sentiment") are saved. The `coalesce(1)` operation is used to consolidate the output into a single file per dataset before writing.
- The processed data is saved in Parquet format to specified output directories (output/cleandata/large/test_data and output/cleandata/large/train_data) using overwrite mode.

9. Creating and Saving Sampled Datasets:

- To facilitate potentially faster hyperparameter tuning, smaller samples are created from the processed data.
- Temporary SQL views are created from the processed DataFrames.
- SQL queries are used to select a balanced sample (180,000 positive, 180,000 negative) from the training data.
- Similarly, balanced samples (20,000 positive, 20,000 negative) are selected from the test data, which are then further split randomly (50/50) into validation and test subsets.
- These sampled training, validation, and test sets (containing "cleaned_text" and "sentiment") are saved in Parquet format using `coalesce(5)` into separate directories (.../train_data_sample, .../val_data_sample, .../test_data_sample).

10. Cleanup:

- Finally, the SparkSession is stopped to release resources (`spark.stop()`).

Model Training

Process Steps

1. Load Data:

- Read the pre-cleaned training, validation, and test datasets (containing text features and sentiment labels) into memory, likely from saved files (e.g., Parquet format).

2. Preprocess Text Data:

- Initialized a `Tokenizer` object from Keras, optionally limiting the vocabulary size (`max_words`).
- Fit the tokenizer on the training text data to create a word-to-index mapping.
- Convert the text data (train, validation, and test sets) into sequences of integers using the fitted tokenizer's `texts_to_sequences` method.
- Pad these integer sequences using `pad_sequences` to ensure they all have a uniform length (`max_len`), suitable for RNN input.

3. Define Model Architecture:

- Instantiated a Keras `Sequential` model.
- Added an `Embedding` layer as the first layer, specifying the vocabulary size (`max_words`) and desired embedding dimension (`embedding_dim`).
- Added a recurrent layer based on the specified `rnn_type` ('simple', 'lstm', or 'gru'), defining the number of neurons.
- Added `BatchNormalization` and `Dropout` layers for regularization.
- Added an intermediate `Dense` layer with a specified activation function (e.g., 'relu').
- Added further `BatchNormalization` and `Dropout` layers.
- Added the final output `Dense` layer with a single unit and a 'sigmoid' activation function, suitable for binary classification.

4. Compile the Model:

- Compiled the constructed model using `model.compile`, specifying the optimizer (e.g., Adam with a defined `learning_rate`), the loss function ('binary_crossentropy' for binary classification), and the metrics to monitor during training (e.g., 'accuracy').

5. Set Up Callbacks:

- Defined Keras callbacks to aid training, such as `EarlyStopping` (to halt training if validation loss doesn't improve for a set number of epochs) and `ReduceLROnPlateau` (to decrease the learning rate if validation loss plateaus).

6. Train the Model:

- Call the `model.fit` method, providing the preprocessed training data (`X_train`, `y_train`).
- Specify the number of training `epochs` and the `batch_size`.
- Include the preprocessed validation data (`X_val`, `y_val`) for monitoring performance on unseen data during training.
- Pass the defined `callbacks` list to the `fit` method. Capture the training history object returned by `fit`.

7. Evaluate the Model :

- After training completes (or stops early), call `model.evaluate` using the preprocessed test data (`X_test`, `y_test`) to obtain the final test loss and test accuracy metrics.

8. Generate Predictions :

- Use the trained model's `model.predict` method on the preprocessed test data (`X_test`). Convert the output probabilities (from the sigmoid activation) into binary class predictions (0 or 1) by applying a threshold (e.g., > 0.5).

Saving the Model

After training and evaluation, the model is saved to enable reuse, for example, in the prediction application. This process is handled by a dedicated function (`save_complete_model`) which utilizes the built-in Keras `model.save()` method. This command saves the entire state of the model, including the network architecture, the learned weights, and the configuration of the optimizer, into a single file in the .keras format. This file is named based on the model type and experiment details and stored in a structured directory. Additionally, a separate JSON file containing metadata such as the model type, save date, and framework version is saved alongside the model file for tracking purposes.

Model Evaluation

Model Tracking

To determine the most effective model architecture and hyperparameters, a systematic experimentation process was undertaken. We explored various configurations, systematically altering parameters across different runs. Key aspects tested included the type of recurrent layer (Simple RNN, LSTM, GRU), the vocabulary size used for tokenization (11092 or 422794 words), the number of neurons in the recurrent layer (32 or 64), the learning rate for optimization (0.001 or 0.005), and the activation function (ReLU or Leaky ReLU). We also experimented with different dataset sizes ('Small' vs. 'Large') and toggled the use of mechanisms like Attention layers and Early

Stopping to prevent overfitting. Each unique combination of these parameters constitutes a distinct experiment. All these experiments, along with their specific configurations and resulting metrics, will be logged using MLflow. The primary metric for comparison will be the *validation accuracy* achieved during training, enabling us to objectively identify the highest-performing model configuration from this series of tests.

Below is the Experiment Sheet

Experiment	TypeOfLayer	VocabCount	NumberOfNeurons	LearningRate	DatasetVolume	Attention	Regularization	EarlyStop	ActivationFunction	Epoch	EmbeddingDimension	Description
EXP-1	Simple	10000	32	0.001	Small	No	Yes	TRUE	relu	15	100	
EXP-2	LSTM	10000	32	0.001	Small	No	Yes	TRUE	relu	15	100	
EXP-3	GRU	10000	32	0.001	Small	No	Yes	TRUE	relu	15	100	
EXP-4	Simple	5000	32	0.001	Large	No	Yes	TRUE	relu	10	150	
EXP-5	LSTM	5000	32	0.001	Large	No	Yes	TRUE	relu	10	150	
EXP-6	GRU	5000	32	0.001	Large	No	Yes	TRUE	relu	10	150	
EXP-7	Simple	5000	64	0.005	Small	No	Yes	TRUE	leaky_relu	15	200	
EXP-8	LSTM	5000	64	0.005	Small	No	Yes	TRUE	leaky_relu	15	200	
EXP-9	GRU	5000	64	0.005	Small	No	Yes	TRUE	leaky_relu	15	200	
EXP-10	Simple	10000	64	0.005	Large	No	Yes	TRUE	leaky_relu	10	200	
EXP-11	LSTM	10000	64	0.005	Large	No	Yes	TRUE	leaky_relu	10	200	
EXP-12	GRU	10000	64	0.005	Large	No	Yes	TRUE	leaky_relu	10	200	
EXP-13	LSTM	10000	64	0.005	Large	Yes	Yes	TRUE	relu	10	150	
EXP-14	LSTM	10000	64	0.005	Small	Yes	Yes	TRUE	relu	15	300	
EXP-0	Simple	10000	32	0.001	Small	No	No	TRUE	relu	15	200	We did a mistake by tokenizing train , val and test with its
EXP-0	LSTM	10000	32	0.001	Small	No	No	TRUE	relu	15	200	own instance. Which effected the model performance. We
EXP-0	GRU	10000	32	0.001	Small	No	No	TRUE	relu	15	200	were getting validation accuracy of < 0.56

Fig 1.0 Experiment Planning

Evaluation Explanation

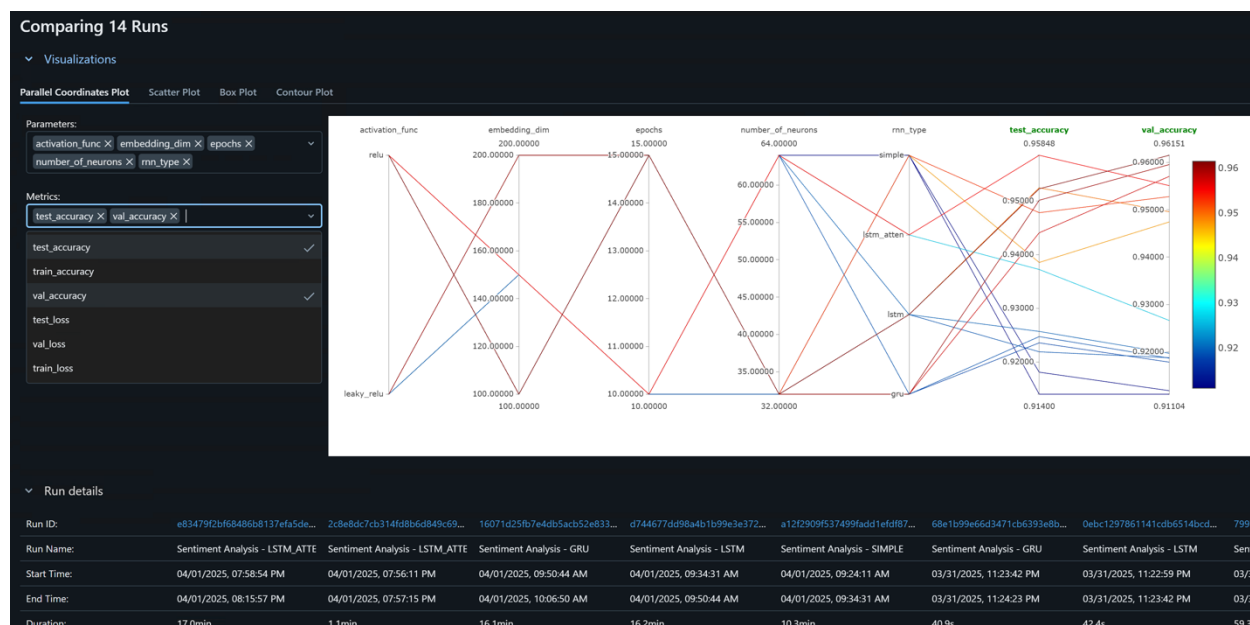


Fig 1.1 Experiment Results

The above plot visualizes the relationship between selected hyperparameters and resulting performance metrics across multiple experiments. Each line represents a single experimental run, connecting its specific parameter values and achieved metrics across the vertical axes. The color of the line corresponds to the validation accuracy, with brighter/warmer colors (like yellow/orange, towards the top of the color bar on the right) indicating higher accuracy (around 0.95-0.96) and darker/cooler colors (blue/purple, towards the bottom) indicating lower accuracy (around 0.91-0.92).

Here are our observations:

1. **Parameters vs. Metrics:** The plot maps `activation_func`, `embedding_dim`, `epochs`, `number_of_neurons`, and `rnn_type` against the `test_accuracy` and `val_accuracy`.
2. **Performance Range:** The validation and test accuracies across these 14 runs range roughly from a low of about 0.91 to a high of approximately 0.96.
3. **Key Parameter Influences:**
 - **Number of Neurons:** There's a clear trend showing that runs using a higher number of neurons (the top value on the axis, likely 64 based on the experiment table) consistently achieved higher validation and test accuracies (warmer colored lines). Runs with fewer neurons (likely 32) generally resulted in lower performance (cooler colored lines).
 - **Embedding Dimension:** Higher embedding dimensions (values like 200 and perhaps 150) strongly correlated with better performance. The lines representing the highest accuracy runs predominantly pass through the upper range of this axis, while lower dimensions (like 100) are associated with lower accuracy lines.
 - **RNN Type:** The plot shows distinct clusters for different RNN types. The highest performing runs (brightest lines) are concentrated on specific points on this axis, suggesting RNN types (LSTM and perhaps GRU, based on run names below the plot and general performance expectations) significantly outperformed others ('Simple' RNN type, which corresponds to some lower-performing runs).
 - **Epochs:** The relationship with epochs is less direct. While the highest accuracy runs tend to cluster around 10 to 15 epochs, there isn't a strict linear correlation. Some runs with high epochs show lower performance, and one high-performing run used around 10 epochs. This suggests an optimal range exists, but the exact number of epochs interacts with other parameters.
 - **Activation Function:** High-performing runs are associated with multiple points on this axis, suggesting other parameters had a more dominant effect and multiple activation functions worked well in combination with the right settings.
4. **Validation vs. Test Accuracy:** The lines generally maintain similar relative heights between the `val_accuracy` and `test_accuracy` axes. This indicates good generalization – models that performed well on the validation set also tended to perform well on the unseen test set. The best runs achieved both high validation and high-test accuracy (~0.96).

Summary

The parallel coordinates plot highlights that using a higher number of neurons (e.g., 64), a larger embedding dimension (e.g., 150-200), and selecting specific RNN architectures (likely LSTM or GRU) were crucial factors in achieving the best validation and test accuracies (around 0.96) in these experiments. The number of epochs showed an optimal range rather than a simple linear trend. The results also showed good consistency between validation and test performance across the runs.

Additional Observations

Based on the comparison charts for the Sentiment Analysis experiments, several key observations can be made regarding model performance across different runs. The dashboard displays test accuracy, test loss, training accuracy, training loss, validation accuracy, and validation loss, allowing for a comprehensive evaluation.

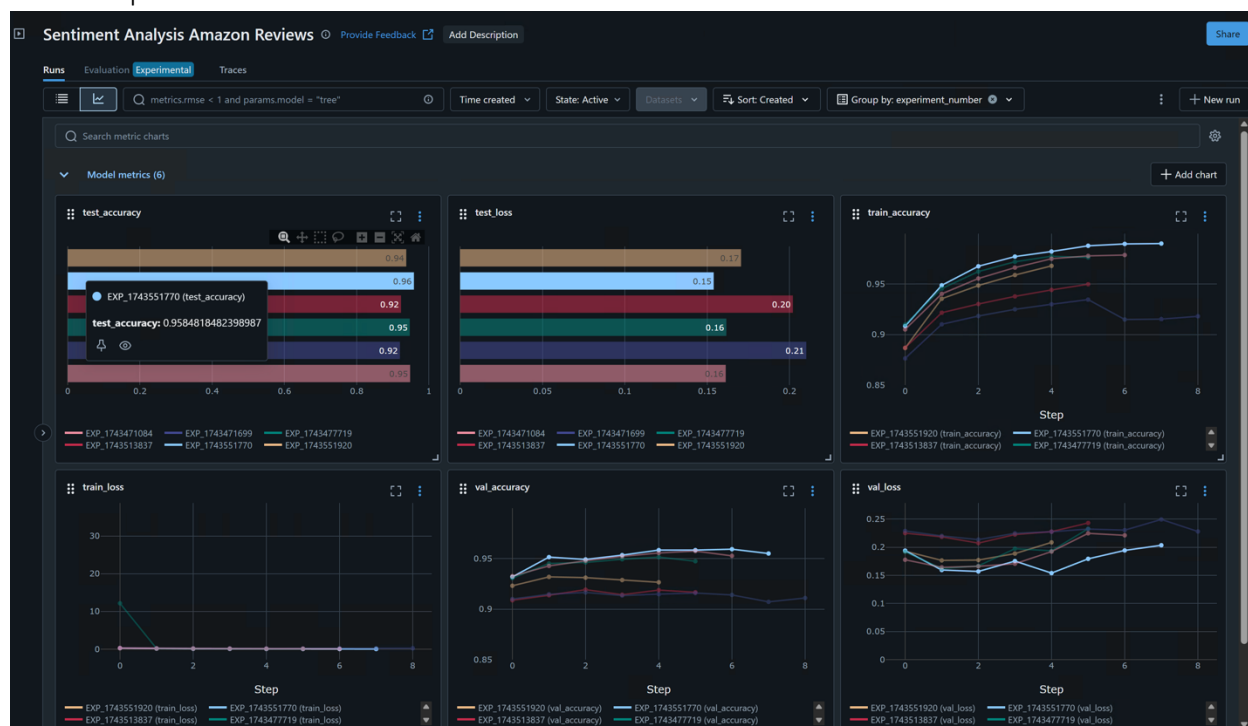


Fig 1.2 Experiment Comparisons Charts

Run EXP_1743551770 demonstrated superior performance, achieving the highest test accuracy of approximately 0.9585, significantly better than other compared runs which showed accuracies around 0.92 or lower. This top-performing run also corresponded to the lowest test loss (around 0.17). The line graphs tracking metrics over training steps reveal expected learning curves: training accuracy generally increased while training loss decreased, eventually plateauing for most runs. Validation accuracy curves showed improvement but with more fluctuations compared to training, while validation loss curves decreased initially but flattened or slightly increased for some runs later in training, potentially indicating the point where overfitting began. These visualizations effectively highlight EXP_1743551770 as the most successful experiment based on its strong generalization to the test set, supported by its performance trends on the validation set throughout the training process.

Confusion Matrix

The below confusion matrix illustrates the performance of the "SIMPLE" RNN model on a binary sentiment classification task (Negative vs. Positive). It shows that the model correctly classified 9043 instances as Negative (True Negatives) and 9024 instances as Positive (True Positives).

However, it made errors by incorrectly classifying 882 Negative instances as Positive (False Positives) and 818 Positive instances as Negative (False Negatives).

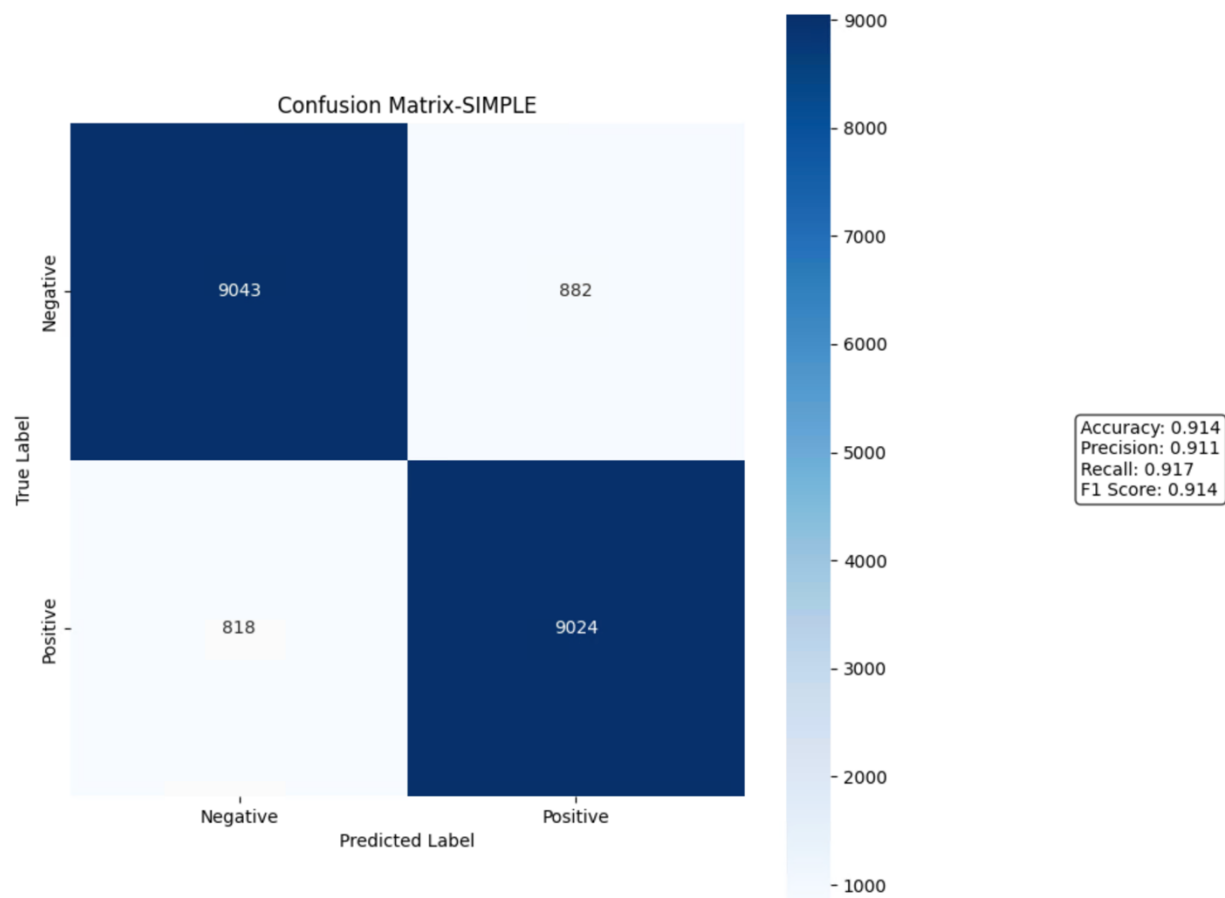


Fig 1.3 Confusion Matrix

Overall, the model achieved an accuracy of 91.4%, meaning it correctly predicted the sentiment for 91.4% of the instances. The Precision of 91.1% indicates that when the model predicted Positive, it was correct 91.1% of the time, while the Recall of 91.7% shows that it successfully identified 91.7% of all actual Positive instances. The F1 Score, which balances Precision and Recall, is also 91.4%, reflecting a reasonably balanced performance between the two types of errors.

Error Analysis

One of our earlier experiments, we used different tokenizer for test, train and validation datasets. Below was our observation.

Below graphs illustrates the negative impact of using inconsistent tokenization across training, validation, and test datasets. While the training accuracy shows a typical learning curve, increasing significantly over the training steps to reach around 0.90-0.91, and the training loss decreases accordingly, the validation performance is drastically different and poor. The validation accuracy

remains very low, fluctuating erratically between approximately 0.52 and 0.53, showing no improvement trend and indicating performance barely better than random chance for a binary classification task. Correspondingly, the validation loss is extremely high and volatile. This major discrepancy arises because fitting separate tokenizers creates different word-to-index mappings for each dataset split.

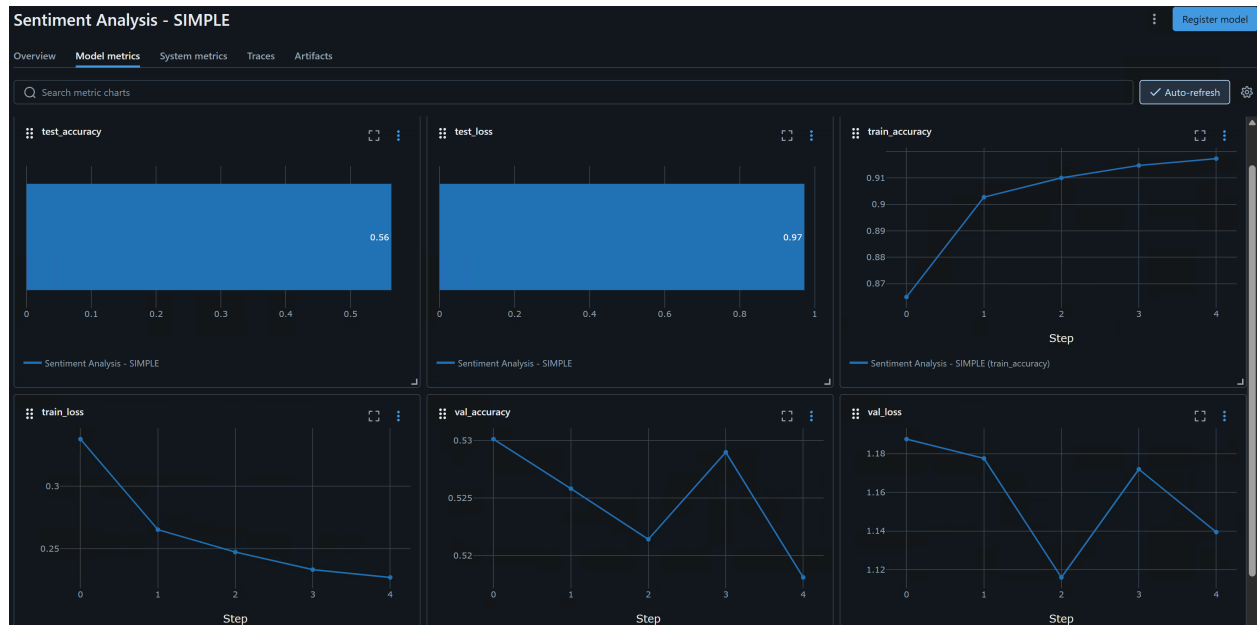


Fig 1.4 Confusion Matrix

Consequently, the model encounters words during validation that were either unknown or mapped to different indices during training, preventing it from generalizing learned patterns and leading to the observed low validation accuracy.

Model Inference

We created a Streamlit application which provides a user interface for sentiment prediction based on a pre-trained Keras model. Users configure the app via a sidebar by providing paths to the Keras model and its corresponding tokenizer JSON file, along with the maximum sequence length for input text (`st.sidebar.text_input`, `st.sidebar.number_input`). While saving the model we are also storing tokenizer in a json format.

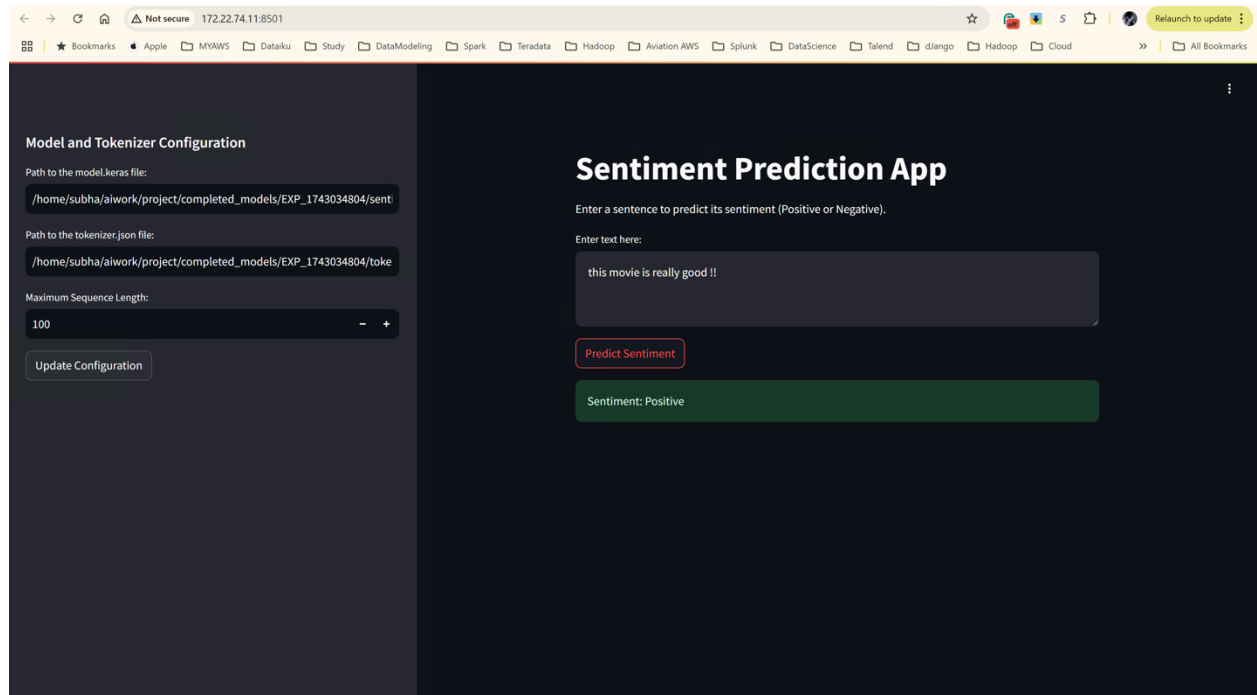


Fig. 1.3 Sentimental Analysis App.

These files are loaded using `tf.keras.models.load_model` and `tokenizer_from_json` via the `load_model_and_tokenizer` function upon clicking "Update Configuration" (`st.sidebar.button`), with the loaded objects stored in session state (`st.session_state`). In the main interface (`st.title`, `st.write`), users input text into a text area (`st.text_area`). Clicking "Predict Sentiment" (`st.button`) triggers the `predict_sentiment` function, which tokenizes the input using the loaded tokenizer (`texts_to_sequences`), pads it to the specified length (`pad_sequences`), and feeds it to the loaded model for inference (`model.predict`). The resulting prediction probability is thresholded at 0.5 to classify the sentiment as Positive (displayed with `st.success`) or Negative (displayed with `st.error`).